

Lost & Found Campus Tracker

A smart, location-aware recovery network for the campus community



M. Harshika

2520090047

Ch. Madhulika

2520090056

B. Neha

2520090013

STEP 1

Project Overview

THE PROBLEM

- ✗ Notices and WhatsApp groups are scattered
- ✗ No record of where items were lost
- ✗ Anyone can claim an item
- ✗ Matching is manual, item by item

OUR SOLUTION

- ✓ One central platform to report lost & found items
- ✓ Map view showing loss hotspots
- ✓ Automatic matching with a confidence score
- ✓ Private-detail ownership verification

TECH STACK

HTML5 · CSS3 · JS

Bootstrap

Spring Boot

REST APIs

MySQL + JPA

STEP 2

Front-End: User Journey

How a user moves through the Bootstrap-based interface

1



Home

Search, recent items,
map preview

2



Login

Role-based access
with validation

3



Report

Lost / found form with
image upload

4



Dashboard

Live status & match
alerts

5



Map

Hotspots of reported
locations

COMMON TO EVERY PAGE

Reusable header/footer • Responsive Bootstrap grid • Inline form validation • fetch() calls to REST APIs

Frontend Modules & Features



Report Lost / Found

Category, description, location, date & photo

Used by: Student, Staff



Search & Filters

Find items by category, location or status

Used by: Everyone



My Dashboard

Track lifecycle status, view match alerts

Used by: Student, Staff



Campus Map

Hotspot view of frequently reported areas

Used by: Everyone



Verification

Enter private details to claim an item

Used by: Claimant



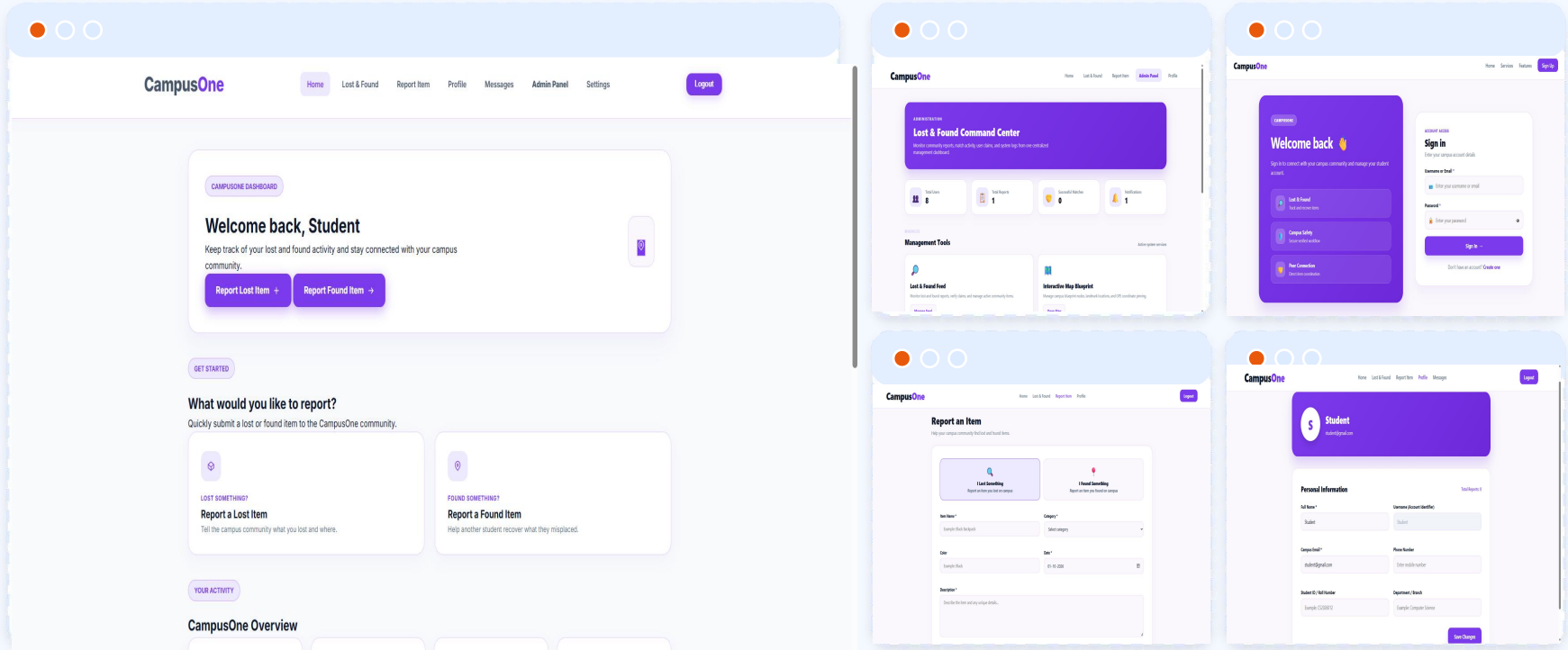
Admin Panel

Review reports, resolve disputes, remove duplicates

Used by: Admin

STEP 4

Frontend Screenshots

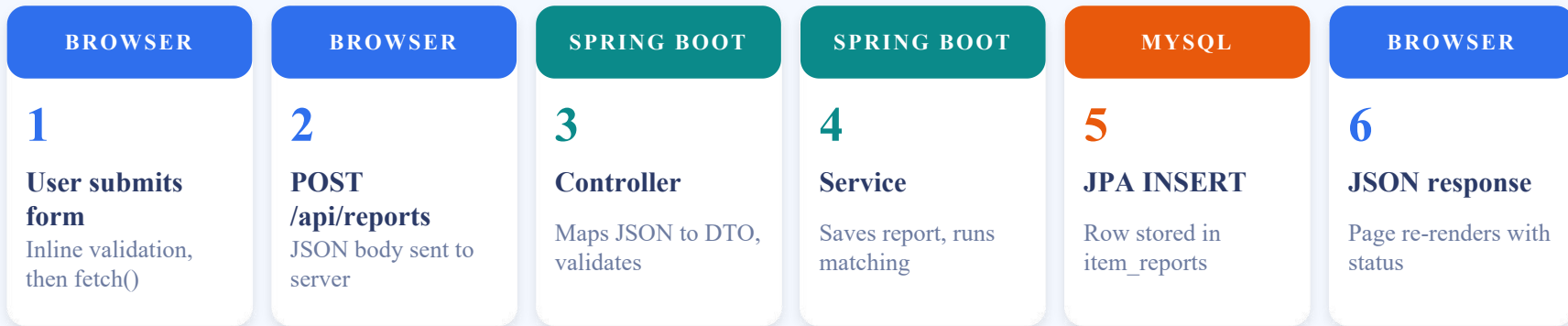


STEP 5

System Architecture: Frontend → Backend → Database



Request Data Flow: Reporting a Lost Item



REQUEST · POST /api/reports

```
{
  "type": "lost",
  "category": "Wallet",
  "location": "Library",
  "userId": 12
}
```

RESPONSE · 201 CREATED

```
{
  "reportId": 58,
  "status": "OPEN",
  "possibleMatches": 2
}
```

Database Design

6

relational tables

PK / FK keys link every table

MySQL via Spring Data JPA

Why a relational design?



Users, reports, matches and verifications are naturally linked records



Constraints keep every report, match and alert valid



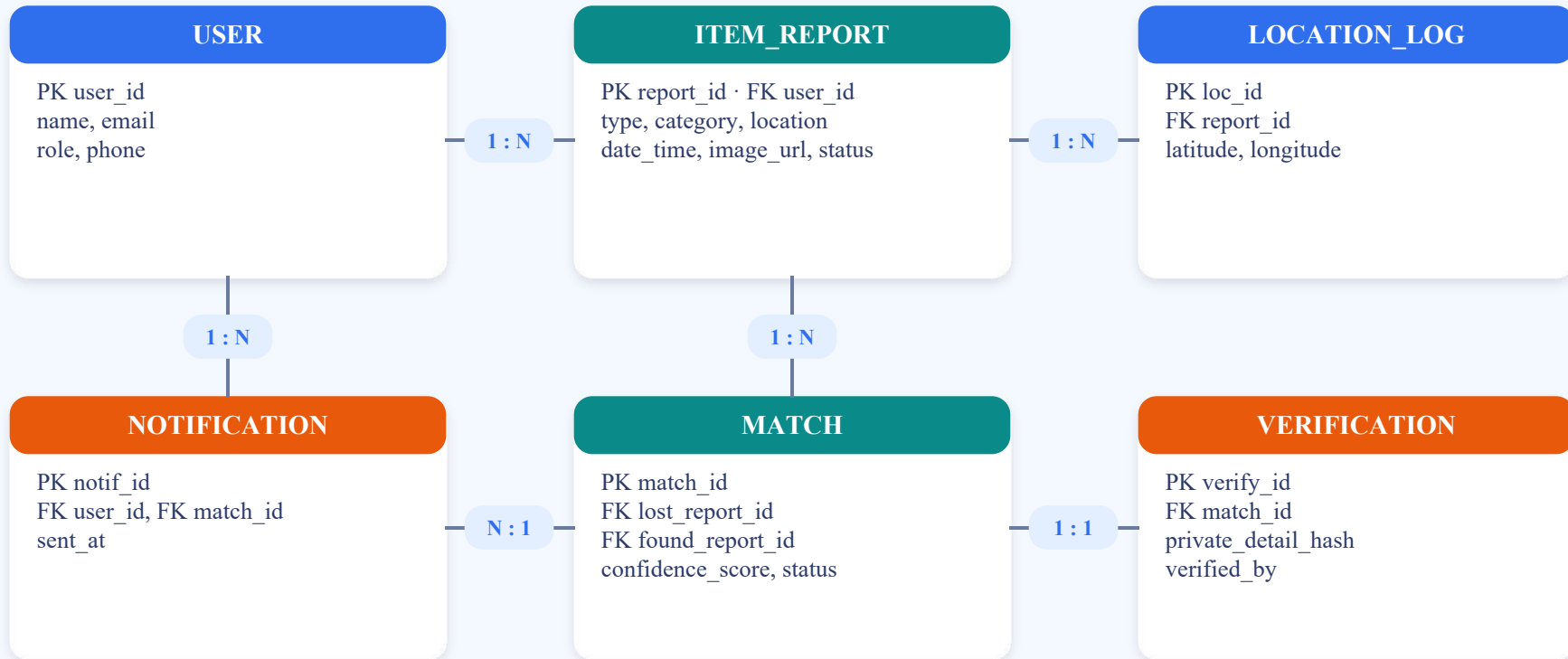
Each fact is stored once and referenced, not copied



Supports joins for matching and hotspot insights

STEP 8

ER Diagram / Database Schema



Tables and Their Purpose



CORE DATA

users

Student, staff & admin accounts

item_reports

Lost / found item details and status



PROCESS

matches

Lost-found pairs with confidence score

verification

Private-detail hash and verifier



SUPPORT

notifications

Email alerts for possible matches

location_log

Latitude / longitude for the map

Keys and Relationships

Primary key

Uniquely identifies a row

Foreign key

Points to another table's primary key

Referential integrity

No match or alert can point to a missing row

FOREIGN KEY → PRIMARY KEY



Database Operations (CRUD)



CREATE

SQL INSERT

REST POST

Submit a lost / found
report



READ

SQL SELECT

REST GET

View items, matches,
dashboard



UPDATE

SQL UPDATE

REST PUT

Change status → Verified
/ Recovered



DELETE

SQL DELETE

REST DELETE

Admin removes duplicate
reports

Sample SQL Queries

1 All reported lost items

```
SELECT * FROM item_reports  
WHERE type = 'lost';
```

2 Each user's reports and status

```
SELECT u.name, r.category, r.status  
FROM users u  
JOIN item_reports r  
  ON r.user_id = u.user_id;
```

3 Best matches first

```
SELECT l.category, m.confidence_score  
FROM matches m  
JOIN item_reports l  
  ON l.report_id = m.lost_report_id  
ORDER BY m.confidence_score DESC;
```

Conclusion and Next Steps

WHAT WE ACHIEVED

- ✓ One platform for campus lost & found
- ✓ Linked relational database with keys and constraints
- ✓ Frontend connected to the backend through REST APIs

NEXT STEPS

- 1 Improve matching accuracy
- 2 Integrate Google Maps hotspots
- 3 Add email match alerts
- 4 More validation & testing
- 5 UI/UX polish



Thank You